Capstone Project

Course: Object Oriented Techniques using Java
Programme: B.Tech (AI / AIML / AI-Twin / AIML-Twin / CS / DS / MNC / CYS / CSE / IT / CSE-Twin / IT-Twin / Integrated M.Tech CSE / IoT)
Semester: III
Assignment Type: Capstone | Project-Based Learning (PBL)
Team Size: 3–5 Students
Total Marks: 40
Evaluation: Continuous Internal Assessment
Plagiarism Limit: ≤ 20%

A. Purpose of the Capstone

This Capstone Project is designed to provide students with an opportunity to integrate and apply all concepts learned in Object Oriented Techniques using Java to develop a complete software solution for a real-world problem.

Students will perform requirement analysis, object-oriented design, software development, testing, and deployment planning while utilizing Java programming concepts such as classes, inheritance, polymorphism, exception handling, multithreading, networking, GUI development, generics, and collections.

The assignment emphasizes:

- Object-Oriented Analysis and Design (OOAD)
- Java-based Software Development
- UML Modeling
- Modular Programming
- Reusability and Maintainability
- Concurrent Programming
- Client-Server Communication
- GUI-Based Application Development
- Data Management using Collections
- Industry-Oriented Software Engineering Practices
- Team Collaboration and Project Management

The objective is to bridge classroom learning with practical software engineering challenges commonly encountered in industry.

B. Case Study Scenario

Title: Design and Development of an Intelligent Java-Based Software Solution for a Real-World Industry Problem

Problem Context

Organizations across industries are increasingly adopting digital solutions to improve operational efficiency, customer experience, decision-making, automation, and resource utilization.

However, many organizations continue to face challenges such as:

- Manual and repetitive processes
- Fragmented information systems
- Poor data organization and retrieval
- Increasing operational complexity
- Limited scalability
- Delayed reporting and decision-making
- Inefficient communication among stakeholders
- Lack of real-time monitoring and tracking
- Data inconsistency and redundancy
- Growing user and transaction volumes

NIET Greater Noida
Get Future Ready An Autonomous Institute

To address these challenges, organizations require intelligent, scalable, and user-friendly software systems capable of automating business processes and supporting future growth.

Student teams are required to select a suitable application domain and design a complete Java-based software solution that addresses a real-world problem using concepts from all five units of the course.

C. Your Task

Students must design, develop, and demonstrate a complete Java-based software solution by applying concepts from all five units of the course.

The project shall be executed in two phases.

Phase I: Object-Oriented Analysis and Core System Design

Objective

Analyze the selected problem and design the core architecture of the proposed software solution.

Unit 1: Basics of Java Programming

Students Must

- Analyze the selected problem domain.
- Identify stakeholders and system requirements.
- Identify classes, objects, attributes, and methods.
- Create UML Class Diagrams and Object Diagrams.
- Apply:
    - Association, Aggregation, Composition, and Generalization
- Implement:
    - Classes, Objects, Constructors, Methods, Control Statements, Console-Based Interaction, and Command-Line Arguments

Deliverables

- Problem Analysis Report
- Requirement Specification Document
- UML Design Document
- Core Java Prototype

NIET Greater Noida
Estd.2001
An Autonomous Institute

Unit 2: OOP Features, Arrays and Lambda Expressions

Students Must

- Implement inheritance hierarchy.
- Apply abstraction through abstract classes.
- Implement interfaces.
- Demonstrate polymorphism through:
    - Method Overloading
    - Method Overriding
- Utilize arrays and jagged arrays for data processing.
- Implement lambda expressions for functional operations.
- Demonstrate code reusability and extensibility.

Deliverables

- OOP Design Report

- Inheritance Structure
- Interface Framework
- Lambda-Based Processing Module

Phase II: Enterprise Application Development and Integration

Objective

Develop a fully functional Java application using advanced Java technologies.

Unit 3: Packages, Exception Handling and String Handling

Students Must

Package Design

Create logical packages such as:

- User Management, Business Logic, Reporting, Utilities, and Data Processing

Exception Handling

Implement:

- Try-Catch Blocks, Multiple Catch Blocks, Nested Try Blocks, Finally Block, Throw and Throws, and User-Defined Exceptions

String Handling

Utilize:

- String, String Buffer, and StringBuilder

for data processing, searching, formatting, and reporting.

Deliverables

- Package Architecture
- Exception Handling Framework
- String Processing Module

Unit 4: Concurrency and I/O Streams

Students Must

Multithreading

Implement concurrent operations relevant to the selected application.

Examples:

- Background processing

NIET Greater Noida An Autonomous Institute Estd.2001 Get Future Ready

- Notifications
- Data synchronization
- Report generation
- Real-time monitoring

Apply:

- Thread Creation
- Runnable Interface
- Synchronization
- Thread Priorities
- Daemon Threads

I/O Streams

Implement:

- Data Storage
- File Reading
- File Writing
- Backup and Recovery

Using:

- Character Streams
- Byte Streams

Socket Programming

Develop client-server communication using:

- TCP Socket Programming

OR

- UDP Socket Programming

Deliverables

- Multithreading Framework
- File Processing Module
- Client-Server Communication Module

Unit 5: GUI Programming, Generics and Collections

Students Must

GUI Development

Develop user-friendly interfaces using:

NIET Greater Noida
Estd.2001
Get Future Ready
An Autonomous Institute

- Swing
- AWT

The application should include:

- Login Interface
- Dashboard
- Data Entry Forms
- Search Screens
- Reports and Analytics

Collections Framework

Apply:

- Array List, LinkedList, HashMap, HashSet, and Tree Set

for efficient data management.

Generics

Implement:

- Generic Classes
- Generic Methods
- Generic Interfaces

to improve code reusability and maintainability.

Deliverables

- GUI Application
- Collections Framework Implementation
- Generics Implementation
- Final Integrated Software System

D. Final Deliverables

Each team shall submit:

Technical Deliverables

1. Approved Problem Statement
2. Requirement Analysis Report
3. UML Diagrams
4. System Architecture Design
5. Java Source Code
6. Package Structure Documentation

NIET Greater Noida An Autonomous Institute Estd 2001 Get Future Ready

7. Exception Handling Documentation
8. Multithreading Design
9. Networking Design
10. GUI Screens
11. Collections & Generics Documentation
12. Testing and Validation Report
13. User Manual

Presentation Deliverables

1. PowerPoint Presentation
2. Software Demonstration
3. Source Code Repository
4. Final Project Report
5. Viva Voce

E. Formatting Guidelines

Report Format

- Font: Times New Roman
- Font Size:
  - Heading: 14 Bold
  - Subheading: 12 Bold
  - Body Text: 12
- Line Spacing: 1.5
- Page Margin: 1 inch on all sides
- Page Numbering: Mandatory
- Report Length: 25–40 Pages

Submission Format

- Soft Copy (PDF)
- Source Code Folder
- Presentation Slides
- Demonstration Video (Optional)

Naming Convention

TeamNo_CourseCode_ProjectTitle.pdf

Example:

Team05_BCSEXXXX_SmartLibraryManagementSystem.pdf

NIET Greater Noida
An Autonomous Institute
Estd 2001
Get Future Ready

F. Team Roles and Responsibilities

To ensure that every evaluation category is adequately addressed, each team member shall take ownership of a specific rubric category while contributing to the overall project.

Team Member	Role	Primary Responsibility
Student 1	Team Lead & Business Analyst	Project planning, requirement gathering, stakeholder analysis, project coordination, milestone tracking, integration reviews, final submission management, project presentation, viva preparation, team coordination support
Student 2	System Architect & Design Lead	UML modeling, class diagrams, object diagrams, architecture design, module decomposition, design pattern selection, object-oriented design validation, project presentation, viva preparation, team coordination support
Student 3	Technical Development Lead	Core application development, business logic implementation, coding standards, feature implementation, module integration, code optimization, project presentation, viva preparation, team coordination support
Student 4	Java Technology & Integration Lead	Implementation of OOP concepts, inheritance, polymorphism, packages, exception handling, multithreading, socket programming, collections, generics, GUI integration, project presentation, viva preparation, team coordination support
Student 5	Quality Assurance, Documentation & Presentation Lead	Testing, debugging, validation, report preparation, user manual creation, PPT development, project presentation, viva preparation, team coordination support

NIET Greater Noida
Get Future Ready
An Autonomous Institute

G. Evaluation Rubric (40 Marks)

36–40: Outstanding | 32–35: Excellent | 28–31: Very Good | 24–27: Good | 20–23: Pass | 0–19: Needs Improvement

Module / Phase	Wt. (%)	MM	Outstanding (91–100%)	Excellent (81–90%)	Very Good (71–80%)	Good (61–70%)	Pass (50–60%)	Needs Improvement (0–49%)
Problem Identification & Requirement Analysis	**10**	**4**	**3.64–4.00** — Exceptional problem definition, stakeholder analysis, scope identification, requirement gathering, and industry relevance. Requirements are comprehensive, measurable,	**3.24–3.63** — Strong problem analysis with clearly defined requirements and minor gaps.	**2.84–3.23** — Good understanding of the problem and requirements with adequate analysis.	**2.44–2.83** — Basic analysis with limited requirement specification.	**2.00–2.43** — Partial understanding of the problem and project objectives.	**0–1.99** — Inadequate problem analysis or missing requirements.

			and well-structured.					
System Design & Solution Architecture	**15**	**6**	**5.46–6.00** — Outstanding system architecture, UML modeling, object-oriented design, modularity, scalability, maintainability, and technical planning.	**4.86–5.45** — Strong architecture and UML design with minor shortcomings.	**4.26–4.85** — Good design and architecture with appropriate object-oriented principles.	**3.66–4.25** — Functional architecture with limited design depth.	**3.00–3.65** — Basic design with several architectural limitations.	**0–2.99** — Poor or incomplete architecture and design artifacts.
Software Development & Technical Implementation	**25**	**10**	**9.10–10.00** — Fully functional solution with excellent implementation quality, modular coding practices, high reliability, and complete feature integration.	**8.10–9.09** — Strong implementation with minor defects and good functionality coverage.	**7.10–8.09** — Good implementation covering most requirements effectively.	**6.10–7.09** — Functional implementation with moderate limitations.	**5.00–6.09** — Basic implementation with several missing features.	**0–4.99** — Incomplete, unstable, or non-functional implementation.
Application of Java Technologies	**15**	**6**	**5.46–6.00** — Exceptional application and integration of Java concepts across the solution with excellent technical depth and best practices.	**4.86–5.45** — Strong utilization of Java technologies with minor gaps.	**4.26–4.85** — Good application of most Java concepts.	**3.66–4.25** — Moderate Java features.	**3.00–3.65** — Limited implementation of Java concepts.	**0–2.99** — Minimal or incorrect application of Java technologies.
Testing, Validation & Quality Assurance	**10**	**4**	**3.64–4.00** — Comprehensive testing strategy including unit testing, integration testing, validation, debugging, and performance verification with documented evidence.	**3.24–3.63** — Strong testing and validation with minor gaps.	**2.84–3.23** — Good testing coverage and validation.	**2.44–2.83** — Basic testing and limited validation.	**2.00–2.43** — Minimal testing evidence.	**0–1.99** — Testing not performed or poorly documented.
Documentation, Presentation, Team Collaboration & Project Management	**25**	**10**	**9.10–10.00** — Professional-quality report, outstanding presentation, effective team coordination,	**8.10–9.09** — Strong documentation, presentation, teamwork, and project planning.	**7.10–8.09** — Good report quality, teamwork, and presentation effectiveness.	**6.10–7.09** — Satisfactory documentation and team participation.	**5.00–6.09** — Limited documentation, quality and team coordination.	**0–4.99** — Poor documentation, weak collaboration, or ineffective presentation.

NIET Greater Noida
Estd.2001
Get Future Ready
An Autonomous Institute

			excellent project management, clear role execution, and exceptional viva performance.					

Estd.2001
Get Future Ready
NIET
Greater Noida
An Autonomous Institute